PLANEJAMENTO TÉCNICO

# Plano de otimização do simulador de treinador

Diagnóstico do frontend, backend e Supabase.
Arquitetura proposta e roteiro completo de implementação.

**Condição obrigatória**
Preservar integralmente o progresso dos usuários atuais. O funcionamento futuro do mundo pode mudar; as decisões e conquistas já registradas devem permanecer.

**Projeto:** DavidCaua14/simulation

**Data da análise:** 31/10/2026

**Referência do código:** c4daca7034b3ad2a7925994569fecafe73b5ed7a

**Escopo:** diagnóstico e planejamento; nenhuma alteração em produção.

# Sumário

1. Objetivo e condição obrigatória . . . 3

2. Diagnóstico confirmado . . . 3

3. Armazenamento encontrado . . . 4

4. Arquitetura recomendada . . . 6

5. Novo escopo de simulação . . . 7

6. Novo comportamento de “próxima partida” . . . 8

7. Mercado e propostas de emprego . . . 10

8. Troca de país . . . 10

9. Consultas, frontend e concorrência . . . 11

10. Melhorias no PostgreSQL . . . 12

11. JSON: onde ajuda e onde atrapalha . . . 13

12. Redução de histórico . . . 13

13. Execução por etapas . . . 15

14. Regras específicas da migração . . . 17

15. Metas e custos . . . 17

16. Segurança e confiabilidade complementares . . . 18

17. Instrução para a IA implementadora . . . 19

## Como usar este documento

As seções 1 a 12 apresentam as evidências e o desenho recomendado. A seção 13 organiza a execução em nove etapas (0 a 8). As seções 14 a 17 definem regras de migração, metas, segurança e a instrução para a IA implementadora. Os valores medidos são referentes à análise de 31/10/2026.

O principal gargalo é o volume de trabalho executado para preparar a próxima partida. A troca para JSON, sozinha, não resolveria isso. A direção recomendada é manter o PostgreSQL, compartilhar os dados de referência e simular com detalhes apenas o contexto relevante para cada treinador. O progresso continua individual e intacto.

# 1. Objetivo e condição obrigatória

Reduzir o tempo de resposta, o armazenamento por carreira e os efeitos negativos do avanço automático, preservando a experiência de gestão de futebol.

**Nenhuma etapa pode reiniciar, reduzir ou recalcular retroativamente o progresso dos usuários atuais.**

Devem ser preservados:

- Conta, personagem, nível, XP, moedas, diamantes, energia e inventário.
- Reputação, credibilidade, títulos, estatísticas e histórico do treinador.
- Clube atual, vínculo, objetivos, situação financeira e compromissos assumidos.
- Elenco, contratações, vendas, contratos, escalação e táticas.
- Condição, lesões, suspensões e satisfação existentes.
- Resultados disputados, campanhas, classificações e conquistas relevantes.
- Propostas, negociações, parcelas e bônus ainda pendentes.
- Partidas em andamento e recompensas já concedidas.

O funcionamento futuro do mundo pode mudar. Isso autoriza reduzir sua simulação, mas não autoriza apagar consequências de decisões anteriores do usuário.

# 2. Diagnóstico confirmado

## 2.1. O custo do avanço

As estatísticas consultadas no PostgreSQL, acumuladas desde 30/10/2026, mostraram:

| Operação | Chamadas | Tempo médio | Maior tempo |
|---|---|---|---|
| career_world_advance | 867 | 3.059 ms | 7.965 ms |
| mgr_hub | 2.127 | 444 ms | 7.160 ms |
| mgr_match_step | 2.521 | 276 ms | 7.862 ms |
| career_player_search | 698 | 348 ms | 6.539 ms |
| Calendário: season_fixtures | 5.957 | 19 ms | 783 ms |

Esses tempos representam execução no banco. Não incluem toda a espera de rede, renderização e encadeamento de chamadas.

O papel authenticated possui limite de 8 segundos por instrução. O maior tempo observado no avanço ficou muito próximo desse limite.

Nos logs das últimas 24 horas apareceram 260 erros de timeout. Também houve registros de encerramento e indisponibilidade de conexões. Não foi estabelecido que todos esses erros vieram do botão de avanço ou do tamanho da base.

### 2.2. Um clique pode executar muitos lotes

O controlador do frontend:

1. Solicita avanço de até sete dias.
2. Continua processando lotes quando o servidor retorna more.
3. Consulta novamente o hub.
4. Repete o processo até alcançar a próxima partida.
5. Permite até 240 iterações na preparação.

Portanto, limitar uma chamada a sete dias não limita o avanço total de um clique a sete dias.

Fonte: worldSimulation.js.

### 2.3. O banco executa serviços globais por dia

A função de produção cw_step_day avança a data e executa:

- Recuperação de condição, ritmo e lesões.
- Atualização de temporadas.
- Finanças.
- Mercado.
- Serviços adicionais de elenco e contratos.
- Propostas de emprego.

cw_simulate_due percorre temporadas ativas do mundo e também cria temporadas de outras regiões.

Além disso, mgr_sim_fixture utiliza o motor detalhado das partidas, incluindo primeiro tempo, segundo tempo, prorrogação e finalização. Isso gera trabalho e registros mesmo em jogos sem participação do usuário.

Conclusão: um clique pode acumular simulação esportiva, financeira, contratual e de mercado de contextos que o usuário nem está acompanhando.

### 2.4. A expiração das propostas acompanha esse avanço

cw_jobs_tick expira propostas conforme a data simulada. O processamento de negociações também considera seus prazos.

O controlador continua avançando sem uma regra geral de parada para decisões urgentes. Assim, uma proposta pode surgir, passar pelo prazo e expirar durante a mesma preparação.

Existe ainda lógica de expiração por horário real em sim_lock_career, quando expires_at está preenchido. É necessário conferir quais modalidades usam cada relógio e evitar expiração indevida de ofertas do modo carreira.

## 3. Armazenamento encontrado

A medição feita durante a análise foi:

| Item | Tamanho aproximado |
|---|---|
| Banco inteiro | 262 MB |
| Tabelas e índices de public | 226 MB |
| Schema cron | 16,9 MB |
| Catálogo interno do PostgreSQL | 16,3 MB |

Principais tabelas, incluindo índices e armazenamento auxiliar:

| Tabela | Tamanho aproximado |
|---|---|
| cw_finance | 28,0 MB |
| season_fixtures | 24,4 MB |
| mgr_matches | 23,6 MB |
| mgr_match_events | 23,1 MB |
| cw_appearances | 16,8 MB |
| cw_contracts | 14,9 MB |
| mgr_players | 14,8 MB |
| mgr_player_stats | 14,8 MB |
| mgr_match_players | 14,5 MB |

Foram encontradas 9 carreiras e 5 mundos inicializados.

O catálogo contém 4.245 atletas e 141 clubes. As estimativas das estatísticas das tabelas indicaram:

- 38.205 registros em mgr_players.
- 21.225 registros em cw_players.
- 178.248 eventos de partidas.
- 64.020 registros de estatísticas de atletas.

Os números de atletas por carreira são consistentes com a cópia de todo o catálogo para cada usuário.

O projeto já possui otimizações de armazenamento nas evoluções 38, 39 e 41. Elas devem ser consideradas na implementação: o problema já recebeu ajustes de retenção, mas continua produzindo muitos dados antes de compactá-los ou removê-los.

## 3.1. A meta de 10 MB

10 MB para o banco Supabase inteiro não é uma meta adequada. A infraestrutura e os catálogos internos já consomem espaço. A documentação informa que projetos novos normalmente começam com dezenas de megabytes. Referência oficial.

Se a intenção for 10 MB de dados dinâmicos por carreira, isso pode ser tratado como orçamento inicial a validar. Não deve ser prometido sem medir o histórico preservado.

Também é necessário distinguir:

- Dados úteis.
- Índices.
- Espaço livre dentro de tabelas após exclusões.
- WAL, que registra alterações e também ocupa disco.

Excluir linhas não garante redução imediata do tamanho físico. VACUUM FULL bloqueia a tabela e não deve ser usado automaticamente durante o jogo. Referência oficial.

# 4. Arquitetura recomendada

## 4.1. Manter carreiras independentes

Cada carreira precisa continuar tendo seu próprio tempo e suas próprias decisões. Usuários diferentes podem contratar o mesmo atleta e estar em temporadas diferentes.

Não transformar essas carreiras em um único mundo dinâmico compartilhado.

Compartilhar o catálogo; individualizar apenas o estado necessário.

O projeto já possui tabelas compartilhadas sim_*. A evolução deve aproveitar essa estrutura e reduzir a criação e atualização de cópias individuais completas.

## 4.2. Separar três tipos de informação

| Tipo | Exemplos | Armazenamento recomendado |
|---|---|---|
| Catálogo compartilhado | Clubes, atletas originais, regras, atributos iniciais | Tabelas de referência versionadas |
| Estado individual | Contratações, dinheiro, contratos, lesões, táticas | Tabelas por carreira |
| Resumos e configurações | Táticas, resumo de partida, metadados | JSONB pequeno e versionado |

A carreira deve apontar para uma versão do catálogo. Atualizações futuras do catálogo não podem substituir silenciosamente os dados dos saves existentes.

## 4.3. Gravar alterações específicas

Para novas carreiras, evitar criar imediatamente registros mutáveis de todos os atletas e clubes.

A leitura pode combinar:

```
Estado efetivo = catálogo da versão do save
               + alterações daquela carreira
```

Materializar informações quando houver necessidade:

- Atleta contratado, vendido ou afetado por uma negociação.

- Clube acompanhado em detalhe.
- Participante de competição ativa que exige estado próprio.
- Lesão, suspensão ou alteração persistente.
- Consequência de decisão do usuário.

Migração dos saves antigos: preservar inicialmente todo o estado existente. Uma linha só pode ser substituída por um valor derivado quando sua equivalência tiver sido comprovada.

Nunca reconstruir os elencos atuais a partir do clube original do catálogo. Isso desfaria transferências.

# 5. Novo escopo de simulação

## 5.1. Separar país de região

Atualmente, BRA e ENG representam grupos de calendário. Eles também abrangem clubes estrangeiros usados em competições continentais.

Exemplo: um clube argentino pode pertencer ao contexto continental associado a BRA, sem ser brasileiro.

Criar conceitos distintos:

- País do clube.
- País ou sistema doméstico da competição.
- Região continental.
- Competições ativas na carreira.
- Escopo do mercado automático.

Usar apenas cw_region para filtrar “meu país” produziria resultados incorretos.

Também considerar clubes galeses participantes do sistema inglês: a participação na liga precisa ser respeitada.

## 5.2. Níveis de simulação propostos

| Contexto | Tratamento |
|---|---|
| Partida do usuário | Motor detalhado atual |
| Rodada doméstica relevante | Resultados dos jogos da rodada, com motor simplificado |
| Competição continental em que participa | Simulação da competição, incluindo clubes estrangeiros |
| Continental após eliminação | Simulação simplificada, preservando chave e classificação |
| Outros países | Sem partidas detalhadas nem mercado automático diário |

Enquanto o treinador estiver no Brasil, a Premier League não deve ser uma dependência para liberar sua próxima partida.

A Libertadores continua funcionando porque é uma competição relevante, mesmo envolvendo clubes de outros países.

### 5.3. Rodadas coerentes

Ao disputar uma rodada do Brasileirão, seus demais jogos devem aparecer concluídos no mesmo contexto da rodada.

“Em paralelo” pode ser uma apresentação simultânea ao usuário. Não exige abrir diversas gravações concorrentes no banco.

Se o processamento precisar de mais de uma chamada:

1. Calcular os jogos restantes com identificadores estáveis.
2. Guardar o andamento de forma recuperável.
3. Publicar a rodada e a classificação quando o conjunto estiver consistente.
4. Retomar uma interrupção sem duplicar resultados.

Uma classificação parcialmente atualizada não deve ser apresentada como rodada concluída.

### 5.4. Países inativos

Países inativos devem ter catálogo consultável, sem executar temporadas detalhadas em segundo plano.

Quando for necessário determinar campeões, classificados ou situação de um país, usar um resumo determinístico, baseado em:

- Força dos clubes.
- Regras de classificação.
- Histórico já consolidado.
- Semente do save.
- Temporada e versão do algoritmo.

Não inventar resultados diferentes cada vez que uma tela for aberta.

Para clubes, países ou competições sem estrutura cadastrada, não oferecer uma experiência de gestão completa que o jogo ainda não suporta.

## 6. Novo comportamento de “próxima partida”

### 6.1. Avançar até o próximo evento relevante

Substituir a preparação orientada apenas a dias por um planejamento de eventos.

O próximo destino deve ser o primeiro entre:

- Partida do usuário.
- Proposta que precisa de decisão.
- Contraproposta recebida.
- Vencimento contratual relevante.
- Registro de transferência.
- Fechamento financeiro necessário.

- Outra decisão urgente configurada.

Exemplo:

```
Hoje: 10/03
Próxima partida: 18/03
Resposta de negociação: 12/03

O clique para em 12/03 e apresenta a resposta.
Após a decisão, o usuário pode continuar até a partida.
```

A data da partida continua realista. O jogo apenas deixa de atravessar decisões sem dar oportunidade de resposta.

## 6.2. Paradas controladas pelo servidor

O servidor deve retornar informações como:

```
{
  "today": "2027-03-12",
  "stop_reason": "decision_required",
  "requires_attention": true,
  "state_version": 48,
  "continuation": null
}
```

O frontend deve interromper a continuação automática ao receber requires_attention.

A proteção precisa existir também no backend. Um cliente antigo não pode continuar avançando por cima de uma decisão protegida.

## 6.3. Prazos das propostas

Manter os prazos e regras existentes, acrescentando:

- Parada ao receber uma proposta que exige decisão.
- Prazo restante visível.
- Nenhuma proposta nova deve expirar no mesmo avanço sem oportunidade de resposta.
- Avanço além do vencimento somente após ação explícita do usuário.
- Mesma regra nas telas, RPCs e rotinas internas.

Não reiniciar o prazo a cada abertura de tela. Isso permitiria prolongá-lo indefinidamente.

## 6.4. Recuperação sem atualizar todos os atletas por dia

Condição e ritmo podem ser calculados pelo tempo decorrido desde a última atualização:

```
condição = mínimo(100, condição anterior + recuperação × dias)
ritmo = máximo(piso, ritmo anterior − redução × dias)
```

Gravar quando o atleta participar de um evento ou sofrer alteração.

A fórmula precisa reproduzir as regras atuais, inclusive arredondamentos, lesões e limites. Efeitos financeiros, contratos e satisfação exigem eventos próprios; não podem ser ignorados porque os dias foram agrupados.

# 7. Mercado e propostas de emprego

## 7.1. Mercado automático local

As decisões automáticas de clubes devem ficar restritas ao escopo doméstico ativo.

Evitar diariamente:

- Percorrer todos os clubes.
- Procurar atletas no mundo inteiro para cada comprador.
- Recalcular informações globais sem mudanças.
- Criar registros de “decisão do dia” para clubes inativos.

Processar o mercado em momentos definidos e com orçamento de trabalho limitado.

## 7.2. Contratação manual internacional

O usuário pode procurar e negociar com atletas estrangeiros.

Isso deve ativar somente as entidades necessárias:

- Atleta.
- Clube vendedor.
- Contrato.
- Negociação.
- Obrigações financeiras relacionadas.

Não é necessário ativar o campeonato do vendedor para responder à proposta.

Preservar as regras existentes de interesse, salário, rivalidade, jogador indispensável, orçamento, janela e limite de elenco.

## 7.3. Empregos no exterior

A implementação atual já utiliza reputação, força dos clubes e credibilidade. Ela também considera condições de contratação internacional.

Transformar o acesso ao exterior em regra explícita e configurável:

```
Elegibilidade = reputação suficiente
              + credibilidade exigida
              + faixa do clube permitida
              + destino suportado pelo jogo
```

Não alterar a reputação dos usuários atuais para fazê-los caber nessa regra.

Não confundir nível da conta, reputação do treinador e credibilidade financeira. Eles têm funções diferentes.

Propostas internacionais já recebidas devem manter sua validade na migração, mesmo que a regra futura se torne mais restritiva.

# 8. Troca de país

Ao aceitar um emprego em outro país:

1. Validar proposta e recursos.
2. Preservar o vínculo e histórico anteriores.
3. Identificar a situação do país de destino na data atual.
4. Reutilizar resultados já existentes.
5. Resumir apenas períodos ainda não resolvidos.
6. Criar o calendário necessário.
7. Ativar o novo contexto doméstico.
8. Suspender a simulação detalhada do contexto anterior.

Não simular retroativamente cada partida de cada dia perdido.

A troca deve preservar:

- Data atual da carreira.
- Histórico do treinador.
- Transferências realizadas.
- Contratos e parcelas existentes.
- Compromissos entre clubes de países diferentes.
- Continuidade das competições relevantes.

Ao voltar a um país, reutilizar seu estado e histórico. Nunca gerar a mesma temporada novamente.

# 9. Consultas, frontend e concorrência

## 9.1. Hub de leitura

O mgr_hub atual chama uma cadeia que chega a mgr_lock. Essa cadeia pode bloquear a carreira, inicializar estruturas e expirar propostas.

career_world_status também chama mgr_lock.

Criar uma consulta de resumo sem efeitos de simulação ou gravação. Abertura de tela e consulta intermediária devem apenas ler o estado.

Inicialização, recuperação de energia e reconciliações devem ter caminhos explícitos, preservando suas regras.

## 9.2. Evitar recarregar o calendário inteiro

useManagerData busca novamente os jogos de todas as competições retornadas pelo hub após atualizações.

Fonte: useManagerData.js.

Recomendações:

- Resumo inicial pequeno.
- Calendário por intervalo ou competição aberta.
- Atualizações das partidas e classificações alteradas.
- Versões para invalidar cache.
- Paginação de mensagens, notícias e históricos.

fetchFixtures não apresenta paginação explícita. Conferir o limite de linhas da API para evitar calendários incompletos quando uma consulta ultrapassar esse limite.

### 9.3. Autoridade e idempotência

Manter decisões e resultados sob controle do servidor.

O projeto já possui UUID por requisição e reutilização desse UUID em tentativas de recuperação. Preservar essa proteção.

Acrescentar uma operação durável de preparação, com:

- Identificador da operação.
- Destino e escopo definidos.
- Versão esperada da carreira.
- Etapas concluídas.
- Resultado persistido.
- Retomada após fechar o navegador.

Duas abas não podem avançar a mesma carreira duas vezes.

A retenção das confirmações de operação deve ser compatível com o prazo de retomada. Não remover um comprovante de idempotência enquanto sua repetição ainda puder causar duplicação.

## 10. Melhorias no PostgreSQL

O Supabase apontou:

| Aviso | Ocorrências |
|---|---|
| Chaves estrangeiras sem índice de apoio | 108 |
| Políticas RLS com oportunidade de otimização | 50 |
| Índices sem uso observado | 12 |

Esses avisos exigem análise. Não criar 108 índices automaticamente: cada índice aumenta armazenamento e custo de gravação.

Priorizar consultas medidas do avanço, negociações, contratos, calendário e finanças.

Procedimento:

1. Identificar as instruções internas mais caras.
2. Examinar planos com dados representativos.
3. Medir leituras, linhas processadas, tempo e gravações.
4. Ajustar consulta ou índice.
5. Comparar antes e depois.

Já existe índice por carreira e data em cw_fixture_dates, e o plano consultado o utiliza. O problema não é simplesmente ausência de índice no calendário.

O rastreamento atual registra principalmente instruções externas; o tempo individual de cada serviço interno ainda precisa ser medido.

Outras prioridades:

- Substituir consultas repetidas por atleta por agregações em lote.
- Atualizar linhas somente quando os valores mudarem.
- Evitar funções caras repetidas em filtros e ordenações.
- Gerar resumos financeiros diretamente para clubes da IA.
- Retirar compactações extensas do caminho de preparação da partida.
- Medir espaço reutilizável antes de qualquer manutenção física.

Para RLS, avaliar o padrão (select auth.uid()) quando adequado, mantendo a autorização por proprietário. Documentação.

# 11. JSON: onde ajuda e onde atrapalha

Não usar um arquivo ou campo gigante contendo todo o save como solução principal.

Isso pode exigir ler e regravar grandes blocos para uma alteração pequena, concentrar conflitos de gravação e dificultar consultas e validações.

| Informação | Escolha recomendada |
|---|---|
| Dinheiro, propriedade e contratos | Tabelas relacionais |
| Negociações e parcelas | Tabelas relacionais |
| Classificação e estatísticas | Tabelas e agregações |
| Táticas e configurações pequenas | JSONB |
| Resumo compacto de partida | JSONB |
| Backup ou exportação do save | JSON versionado e comprimido |

O próprio Supabase suporta JSONB para estruturas flexíveis. Isso complementa as tabelas relacionais. Documentação.

# 12. Redução de histórico

Prevenir a geração de dados desnecessários antes de depender de exclusões posteriores.

Para partidas da IA, guardar preferencialmente:

- Placar.
- Resultado do confronto.
- Dados necessários à classificação.
- Estatísticas agregadas.
- Disciplina necessária ao desempate.

- Resumo compacto para consulta.

Não produzir centenas de eventos detalhados para depois removê-los.

Preservar registros necessários a:

- Partidas do usuário.
- Conquistas e estatísticas.
- Salários e bônus por atuação.
- Cláusulas de desempenho.
- Suspensões.
- Critérios de desempate.
- Auditoria financeira.

A evolução 41 já documenta a necessidade de manter cartões para fair play. O novo motor simplificado deve preservar essa regra.

Não apagar ledgers financeiros, recompensas ou registros de idempotência indiscriminadamente para atingir uma meta de tamanho.

# 13. Execução por etapas

Cada etapa deve ser entregue separadamente, com alterações revisáveis, evidências e possibilidade de recuperação.

## Etapa 0 — Preservação e linha de base

Entregas:

- Backup consistente e restauração validada em ambiente isolado.
- Inventário das funções realmente instaladas.
- Mapeamento entre código de produção e arquivos SQL.
- Matriz dos campos protegidos por carreira.
- Medição do clique completo: tempo, chamadas, dias e registros alterados.

Aceite: saves antigos restaurados com progresso equivalente e plano de comparação pronto.

Não executar scripts de testes que alteram carreiras contra produção. Alguns testes existentes modificam reputação, propostas e outros estados.

## Etapa 1 — Impedir perda de decisões

Entregas:

- Parada no servidor para propostas e decisões urgentes.
- Respeito à parada no frontend.
- Unificação das regras de expiração do modo carreira.
- Mensagem clara do motivo da interrupção.

Aceite: uma proposta recebida durante a preparação não expira sem oportunidade de resposta.

Esta etapa vem antes da reestruturação ampla do armazenamento.

## Etapa 2 — Tornar consultas leves

Entregas:

- Hub sem gravações ou inicializações.
- Resumo intermediário pequeno.
- Calendário carregado conforme necessidade.
- Atualização por versão ou por alterações.

Aceite: consultar ou abrir uma tela não avança o mundo, não expira propostas e não dispara inicialização global.

## Etapa 3 — Restringir a simulação ao contexto ativo

Entregas:

- Modelo explícito de país, competição e região.
- Países inativos fora do avanço diário.
- Mercado automático local.
- Tratamento continental independente.
- Elegibilidade internacional configurada.

Aceite: uma carreira no Brasil prepara jogos domésticos e continentais relevantes sem processar a liga inglesa.

## Etapa 4 — Simplificar partidas da IA

Entregas:

- Motor resumido para partidas sem intervenção humana.
- Resultados determinísticos por partida.
- Rodadas consistentes.
- Estatísticas, disciplina e classificação preservadas.
- Continuação leve após eliminação continental.

Aceite: a rodada conclui corretamente sem usar o motor detalhado em todos os jogos da IA.

## Etapa 5 — Reduzir cópias por carreira

Entregas:

- Catálogo versionado.
- Estado individual por alterações.
- Materialização sob demanda.
- Recuperação por tempo decorrido onde houver equivalência.
- Adaptadores para os formatos usados pelas telas.

Aceite: novas carreiras deixam de criar cópias mutáveis completas do catálogo sem necessidade.

## Etapa 6 — Migrar saves existentes

Entregas:

- Migração idempotente e retomável por carreira.
- Conferência do progresso antes e depois.
- Preservação das negociações e obrigações existentes.
- Ativação gradual por carreira.
- Recuperação compatível com dados já gravados pelo modelo novo.

Aceite: nenhuma diferença nos campos protegidos e continuidade do save na mesma data e situação.

Não basta desligar uma flag para voltar ao motor antigo: ele pode depender de dados que o modelo novo deixou de gerar. A estratégia de recuperação precisa considerar isso.

## Etapa 7 — Índices, retenção e manutenção

Entregas:

- Índices justificados por medições.
- Consultas internas otimizadas.
- Retenção por finalidade.
- Política para históricos técnicos.
- Manutenção física planejada, se necessária.

Aceite: redução comprovada de trabalho e crescimento controlado, sem perda funcional.

### Etapa 8 — Validação e liberação gradual

Validar:

- Carreira iniciante e avançada.
- Brasil e Inglaterra.
- Participação e eliminação continental.
- Desemprego.
- Mudança e retorno de país.
- Negociação internacional.
- Virada de mês e temporada.
- Partida em andamento.
- Duas abas.
- Timeout, repetição de requisição e navegador fechado.

Liberar primeiro para um conjunto pequeno de carreiras, observar e ampliar.

## 14. Regras específicas da migração

1. Não recriar a carreira.
2. Não voltar a data para o início da temporada.
3. Não recalcular partidas disputadas pelo usuário.
4. Não reconstruir elencos pelo catálogo original.
5. Não converter ausência de simulação estrangeira em derrotas ou punições.
6. Não conceder XP, moedas ou títulos pessoais por resumos automáticos do exterior.
7. Não cancelar propostas anteriores apenas porque o novo filtro internacional mudou.
8. Não migrar uma partida ao vivo no meio do motor: adiar a migração dessa carreira ou manter sua execução na versão anterior.
9. Não excluir estado antigo antes da validação do novo.
10. Não aumentar níveis, alterar economia ou rebalancear progressão como parte desta otimização.

As comparações devem ocorrer por carreira, com controle de concorrência. Comparar hashes globais enquanto outros usuários jogam pode gerar divergências legítimas.

O gerador atual de releases possui um modo live que pula comparações de impressões digitais. Esse modo, sozinho, não comprova preservação de progresso.

Fonte: build-editor-release.mjs.

## 15. Metas e custos

Metas iniciais propostas, ainda não demonstradas:

| Métrica | Meta de validação |
|---|---|
| Feedback visual do clique | Até 100 ms |

| Métrica | Meta de validação |
|---|---|
| Hub leve, p95 | Até 300 ms no servidor |
| Preparação comum, p95 | Até 1 segundo no servidor |
| Preparação comum | Uma chamada de avanço; continuação excepcional |
| Propostas perdidas sem oportunidade de decisão | Zero |
| Divergência de progresso na migração | Zero |
| Partidas detalhadas em países inativos | Zero |
| Crescimento por temporada | Medido e limitado por categoria |

Essas metas devem ser verificadas com rede móvel, carreiras avançadas e usuários simultâneos. Não são promessa de resposta instantânea.

Como referência de escopo, o catálogo possui 141 clubes, dos quais 40 são brasileiros, e 4.245 atletas, dos quais 1.355 estão em clubes brasileiros. Restringir serviços domésticos reduz muito o universo processado, mas competições continentais e negociações internacionais acrescentam entidades.

Essas proporções não representam uma previsão de redução total do banco ou da latência.

O maior custo da mudança será de desenvolvimento e validação: dependências entre calendário, contratos, mercado e estatísticas precisam ser preservadas.

O custo operacional tende a cair com menos partidas, gravações e consultas. Não foi calculada uma economia mensal em reais; isso exige plano contratado, uso, concorrência e medições após um protótipo.

# 16. Segurança e confiabilidade complementares

Os avisos de segurança incluíram funções SECURITY DEFINER, funções com search_path ajustável e tabelas com RLS sem políticas.

Esses avisos não comprovam, isoladamente, uma vulnerabilidade: várias tabelas podem ser acessíveis somente por RPCs controladas. A revisão deve verificar:

- Identidade e propriedade da carreira em toda operação.
- Permissões de execução das funções internas.
- search_path fixo nas funções privilegiadas.
- Isolamento entre usuários.
- Chaves privilegiadas fora do frontend.

Não trocar todas as funções para SECURITY INVOKER nem criar políticas genéricas automaticamente.

Referências dos avisos: funções privilegiadas, search path e RLS sem políticas.

O token fornecido na conversa deve ser revogado e substituído. Não incluí-lo em documentos, commits ou arquivos públicos. Nenhum valor de credencial está incluído neste documento.

# 17. Instrução para a IA implementadora

Implemente este plano em etapas independentes, começando pela preservação dos saves e pelas paradas de decisões urgentes.

Antes de cada etapa, confira as definições realmente instaladas no Supabase e identifique suas dependências. Os arquivos SQL possuem sucessivas substituições e wrappers; não use uma definição antiga isoladamente.

Preserve integralmente o progresso atual. Não reinicie carreiras, não reconstrua elencos originais, não altere níveis e não recalcule resultados do usuário.

Mantenha o servidor como autoridade. Preserve idempotência, isolamento por carreira e continuidade após falhas.

Para cada entrega, apresente: problema resolvido, arquivos e funções envolvidos, migração, validação de preservação, medições e procedimento de recuperação.

Não execute testes destrutivos ou simulações de diagnóstico em produção. Antes de qualquer limpeza, demonstre quais regras e telas dependem dos dados.

Priorize reduzir o trabalho gerado por clique e por temporada. Valide tamanho, latência e experiência separadamente.

## 17.1. Registro de validação da análise

Código e definições de produção inspecionados, métricas e logs consultados, suíte existente worldSimulation.test.mjs aprovada. Não foi executada uma simulação de carreira em produção, nem foram aplicadas alterações.

Este PDF reproduz o plano apresentado na conversa. A conversão para PDF não representa uma nova auditoria, não atualiza as medições e não executa as etapas propostas.